Claude Code Practical Engineering Cheatsheet

CLI • prompting • skills • context • subagents • hooks • MCP • CI • enterprise security

Verified: 1 November 2026. Designed for developers, tech leads and platform engineers. Built from the attached user-supplied reference and checked against Anthropic's official documentation. Commands change quickly: confirm the local version with `claude --version`, available commands with `/help`, and flags with `claude --help`. Superscript numbers link to sources at the end.

1 Operating model: reasoning vs enforcement

Claude Code is a coding agent with a context-aware agent loop: inspect repo, reason, call tools, edit, test, iterate. The *model proposes* actions; the *runtime/harness* executes and can constrain them.[1,2]

Prompt or `CLAUDE.md` — Advisory instructions; high flexibility; not a hard safety boundary
Skill `SKILL.md` — Reusable, on-demand task procedure; model interprets
Subagent — Isolated task context; tool/model restrictions; model still reasons
Hook — Executes on a named lifecycle event; can block and log
Permission rules — Runtime `deny > ask > allow`; deterministic access control
Sandbox / OS — Process-level filesystem/network containment for supported tools

Practice: Use models for exploration, synthesis and implementation; use tests, rule engines, permissions, hooks and human approvals for must-pass conditions. An agent cannot make high-impact decisions reliable merely by repeating instructions.

2 Install, start and authenticate

Preferred native installers (macOS/Linux/WSL; Windows PowerShell):[3]

```
curl -fsSL https://claude.ai/install.sh | bash
# Windows PowerShell:
irm https://claude.ai/install.ps1 | iex
# Alternative, Node.js >=22:
npm install -g @anthropic-ai/claude-code
claude --version
claude doctor
claude auth login
```

Daily flow: `cd repo && claude` → `/init` (first time) → inspect `CLAUDE.md` → ask a bounded task → test → `/diff` → commit.[1,3]

```
claude                          # interactive
claude "explain the payment module"
claude -c                       # continue recent
claude -r "auth-refactor"       # resume by name/ID
claude update                   # native update
claude auth status              # JSON auth state
```

Guardrail: Review an installer script before piping it to a shell in regulated environments. Use enterprise-approved install channels, pinned versions and managed credentials.

3 CLI and headless automation

Common flags (see exact version requirements in the official CLI reference).[4]

`-p, --print` — Non-interactive task and exit
`--output-format json` — Machine-readable result (print mode)
`--output-format stream-json` — Incremental event stream
`--json-schema` — Validate final structured output against JSON Schema
`--max-turns N` — Cap agent turns in print mode; error if exceeded
`--max-budget-usd N` — Budget limit in supported print-mode configurations
`--model <id/alias>` — Choose model for session
`--effort <level>` — Model reasoning effort (availability varies)
`--permission-mode plan` — Start read-only planning
`--allowedTools ...` — Pre-approve listed tool patterns; **not** an exhaustive allowlist
`--tools "Read,Bash"` — Restrict available built-in tools
`--disallowedTools ...` — Deny named tool patterns, including MCP
`--mcp-config file` — Load MCP configuration
`--add-dir dir` — Additional accessible working directory
`--bare` — Minimal fast scripting mode; skip auto-discovery of memory, skills, hooks, MCP, etc.

```
cat build.log | claude -p \
  "Diagnose root cause; return fixes" \
  --output-format json --max-turns 8

claude -p "Summarize changed files" \
  --tools "Read,Bash" \
  --permission-mode dontAsk \
  --max-turns 5
```

Guardrail: `--allowedTools` only removes prompts for matched actions. To *limit capabilities*, use `--tools`, deny rules, sandboxing and infrastructure policy. `--dangerously-skip-permissions` is for disposable, strictly isolated environments only.
[4,7]

4 Interactive command map

Session / context:[5]

`/help`, `/status`, `/doctor` — Discover features, environment, diagnostics
`/init`, `/memory` — Bootstrap/edit repo memory
`/clear` vs `/compact` — Fresh task vs summarize current session
`/context`, `/usage`, `/cost` — Context composition and usage
`/model`, `/effort` — Select model and reasoning budget
`/plan` — Enter planning mode
`/rewind`, `/diff` — Restore checkpoint; inspect changes
`/branch`, `/resume` — Explore alternative; restore session

Build, verify, scale:[5,6]

`/run`, `/verify` — Run app and verify behavior (bundled skills)
`/simplify` — Review changed code for reuse/quality
`/code-review [--fix]` — Bug-focused diff review; optional fixes
`/security-review` — Security-focused diff assessment
`/batch <task>` — Break large changes into worktrees/PRs
`/goal <condition>` — Iteration toward stated completion condition
`/tasks`, `/background` — View tasks; detach session
`/btw <question>` — Ephemeral side question
`/mcp`, `/plugin`, `/skills` — Connections, extensions, available skills
`/permissions`, `/sandbox` — Runtime access and isolation

Practice: Availability may vary by version, plan, OS and feature rollout. Slash commands may be built-ins, bundled *skills*, or workflows—do not assume identical execution guarantees.

5 Keyboard and prompt shortcuts

`Shift+Tab` — Cycle permission modes (terminal-dependent)
`Ctrl+C` / `Esc` — Interrupt current work
`Ctrl+O` — Expand transcript / tool details
`Ctrl+G` — Edit prompt in external editor
`Ctrl+S` — Stash / restore draft
`Ctrl+R` — Search prompt history
`Ctrl+L` — Redraw terminal
`Ctrl+B` — Background task
`Ctrl+X Ctrl+K` — Stop background subagents (confirm twice)
`@path/to/file` — Reference repo file
`@agent-name` — Invoke selected custom subagent
`!command` — Shell command mode; outputs visible to Claude

Practice: Use `@file` for exact context instead of pasting huge files. Paths and image-paste keybindings can be terminal-specific.

[8]

6 Prompt engineering: specification + proof

Template: *Outcome + scope + constraints + reference + executable verification + output format.*[1,9]

```
Goal: Add idempotent retries for payment capture.
Scope: src/payments/; do not modify API contracts.
Reference: follow patterns in @src/orders/retry.ts.
Constraints: bounded retries; no duplicate charge;
  preserve audit events; no secrets in logs.
Verify: unit + integration tests, lint and types.
Done when: tests pass and replay test proves
  same idempotency key cannot double-charge.
Output: files changed, tests run, remaining risks.
```

Task strategies:

- **Explore:** Ask it to trace callers and risks before touching code.
- **Plan:** Use `/plan` for cross-module edits, migrations and public APIs.
- **Execute:** Implement the agreed vertical slice; keep diffs bounded.
- **Prove:** Run executable tests and show evidence, not assurance.
- **Review:** Independent code/security review; inspect git diff.

Guardrail: Do not equate *"Task complete"* with correctness. Require reproducible commands, expected/actual output and known limitations.

7 Memory and context engineering

Choose the right location:[10,1]

`./CLAUDE.md` — Repository-shared instructions (team)
`./CLAUDE.local.md` — Git-ignored personal instructions
`/.claude/CLAUDE.md` — Personal global instructions
`.claude/rules/*.md` — Modular, often path-scoped policy/conventions
`.claude/skills/*/SKILL.md` — Demand-loaded reusable procedure
`.claude/agents/*.md` — Specialized subagent profiles
`.claude/settings.json` — Project-level permissions/hooks
`.claude/settings.local.json` — Per-user project settings
`/.claude/settings.json` — User-wide settings

Recommended `CLAUDE.md` skeleton:

```
# Project invariants
- TypeScript strict mode. Node 22.
- Never silently change public API schemas.
# Commands
- Install: pnpm install --frozen-lockfile
- Test: pnpm test; Types: pnpm tsc --noEmit
# Design decisions
- Idempotency is checked in domain service.
- Write business audit events on each transition.
# Done
- Add tests and report command results.
```

Path-scoped rules: `.claude/rules/api.md`

```
---
paths:
  - "src/api/**/*.ts"
---
- Validate every external input.
- Keep response contracts backward compatible.
```

Practice: Aim for fewer than about 200 lines in `CLAUDE.md`; retain non-obvious facts, conventions and commands. Move detailed procedures to skills and area rules. Do not hard-code speculative 70%/90% context reset thresholds; use `/context` and task boundaries.
[1,10]

8 Skills: reusable procedures in depth

Use when: You repeatedly issue the same multi-step procedure (migration, reviews, incident response, release, domain design). Skills carry task instructions and supporting files; descriptions are discoverable, full bodies load on invocation. They are interpreted by the model, *not* guaranteed execution controls.[11] **Where:** `.claude/skills/<name>/SKILL.md` (project) or `/.claude/skills/<name>/SKILL.md` (personal). Invoke `/<name> [args]`; Claude can trigger an eligible skill by matching its description.[11]
Example: `.claude/skills/fix-issue/SKILL.md`

```
---
name: fix-issue
description: Fix a tracked bug with a reproducible
  test and minimal safe patch.
disable-model-invocation: true
allowed-tools: Read Grep Glob Bash Edit
---
Issue: $ARGUMENTS
1. Reproduce the failure; capture evidence.
2. Trace the smallest responsible code path.
3. Add a failing regression test.
4. Implement a minimal fix; preserve APIs.
5. Run tests, lint, type checks and git diff.
6. Report root cause, patch and residual risk.
```

`name`, `description` — Command identity and discovery
`disable-model-invocation: true` — User must trigger; for side-effect procedures
`user-invocable: false` — Background reference skill, not user command
`allowed-tools` — Pre-approve matching tool uses for the skill; not a sandbox
`context: fork` — Run isolated subagent context; task must be self-contained
`agent: Explore` — Select agent for forked skill
`$ARGUMENTS` — Text passed after skill name

Design / evaluation practices:

1. Write a *trigger description* containing the actual problem vocabulary; avoid broad descriptions like "do coding."
2. Put only the workflow in the main `SKILL.md`; add `references/`, `scripts/`, `assets/` for detail.
3. State preconditions, execution sequence, failure handling and observable acceptance criteria.
4. Make destructive workflows manual-invocation only, and keep permissions separately enforced.
5. Test with positive triggers, near-miss negatives, ambiguous tasks, missing input and adversarial repository content.
6. Measure quality: activation precision/recall, task success, tool calls, tokens, elapsed time, safety violations.

Practice: Start a skill from a successful repeated prompt. Run `/skills` to audit available skills and `/doctor` for waste/conflicts.
[11]

9 Subagents and parallel work

Choose a subagent for bounded context, specialized review or a distinct tool/model profile; choose separate worktrees for concurrent code writing. Child contexts do not automatically inherit all of the parent discussion.[12] **Example:** `.claude/agents/security-reviewer.md`

```
---
name: security-reviewer
description: Review changed code for auth, injection,
  secrets, data leakage and unsafe side effects.
tools: Read, Grep, Glob, Bash
model: inherit
---
Only review; never modify files. Inspect git diff.
Return finding, severity, file:line, exploit path,
proposed mitigation and missing tests.
```

Invoke: `@security-reviewer review auth changes` (choose agent from picker), or ask in natural language. Use `claude --agent security-reviewer` to run a whole session under a profile.[12] **Advanced:** Add `isolation: worktree` to a coding subagent to get separate Git workspace; `model: haiku|sonnet|opus|inherit` (based on availability); `maxTurns` to bound loops; `skills` to preload domain knowledge.[12]

Subagent — One task, summarized results to parent; context isolation
`/batch` — Large independent refactor units, worktrees and PRs
`/tasks` — Inspect background jobs/subagents
`/background` — Detach full running session
Agent teams — Cooperative sessions with shared task planning (feature-dependent)

Guardrail: Parallelism multiplies model usage and increases integration conflicts. Isolate filesystem writes; allocate non-overlapping files/ownership; reconcile and rerun full tests on integration branch.

10 Hooks: deterministic lifecycle controls

Hooks are event-driven commands (or HTTP/prompt/agent handlers) configured in settings or eligible extensions. Invocation is triggered by harness events; a hook's external script must enforce the condition correctly.[13]

`PreToolUse` — Block protected writes or unsafe tool calls
`PostToolUse` — Format/test affected files, observe result
`PostToolBatch` — Record results after parallel calls
`Stop` — Check completion; report failed verification
`SessionStart` — Load concise dynamic context
`PreCompact`/`PostCompact` — Snapshot/restore essential constraints
`PermissionDenied` — Observe denied action, route safe retry
`ConfigChange` — Audit settings/skills changes
`SubagentStart`/`SubagentStop` — Lifecycle instrumentation

Minimal blocking hook: project `.claude/settings.json`:

```
{
 "hooks": {
  "PreToolUse": [{
   "matcher": "Edit|Write",
   "hooks": [{
    "type": "command",
    "command": "python3 .claude/hooks/block_protected.py"
   }]
  }]
 }
}
```

Handler: hook receives JSON on `stdin`; for `Edit`/`Write` inspect `tool_input.file_path`; write a reason to stderr and `exit(2)` to block. Working example and tests are in the packaged `examples/` directory.[13]

Guardrail: An Edit/Write hook does *not* cover writes via Bash, MCP or independent tools. Combine deny rules, sandbox and OS boundary. A Stop hook should guard against recursion and excessive retries. Treat hook scripts as trusted executable code; review changes in PR.

11 Permissions, sandbox and safe autonomy

Modes (names are API/CLI values):[7]

`default` **(Manual)** — Prompt for actions needing permission
`auto` — Classifier checks actions without routine prompts
`acceptEdits` — Auto-approve eligible edits; other tools may prompt
`plan` — Read-only exploration and planning
`dontAsk` — Deny anything that would otherwise prompt
`bypassPermissions` — Bypass prompts; dangerous outside isolation

Prefer declarative permissions: `.claude/settings.json`:

```
{
 "permissions": {
  "allow": ["Bash(pnpm test)",
            "Bash(pnpm tsc --noEmit)"],
  "ask": ["Bash(git push *)"],
  "deny": ["Read(./.env)",
           "Read(./secrets/**)",
           "Bash(git push --force *)"]
 }
}
```

Enforcement order: `deny > ask > allow`, regardless of how specific a rule looks. `auto` is an additional classifier, not a deterministic substitute. Sandbox with `/sandbox`; configure filesystem/network boundaries, and use strict mode where appropriate.[7,14]

Enterprise baseline:

- Centrally managed settings for prohibited operations, MCP allowlists and data-boundary requirements.
- Ephemeral development credentials; secret scanner; masked logs and egress restrictions.
- Branch protection, signed commits if required, CI tests, PR reviewers and mandatory approval for production changes.
- Treat repository text, web pages, issues, MCP results and logs as *untrusted input*; never let them override tool policy.

Guardrail: A single deny rule does not protect data from scripts with indirect file access. For high-consequence codebases, isolate workloads at the OS/container and credential layers.[1,14]

12 MCP, tools and plugins

MCP connects tools/resources from remote HTTP servers and local stdio processes. Use only vetted providers and least-privilege credentials. `.mcp.json` at repo root can define team-shared servers, subject to workspace approval.[15]

```
# HTTP (recommended for most remote servers)
claude mcp add --transport http docs \
  --scope project https://mcp.example.com/mcp
# Local stdio process
claude mcp add --transport stdio localtools \
  -- python3 tools/server.py
claude mcp list
claude mcp get docs
claude mcp remove docs
# Interactive: /mcp
```

Design constraints: expose purpose-built, read-only and typed tools where practical; separate lookup from action; validate schema and authorization at the server; log calls and approval decisions; do not expose hundreds of overlapping tools to the model.[15,1] **Plugins** package skills, hooks, subagents, MCP and related components. Inspect sources and permissions before using external plugins; install via `/plugin` UI. Avoid placing API keys in committed `.mcp.json`.[16]

13 Workflows: copyable engineering prompts

Bug with regression:

```
Reproduce failure in @tests/payments.test.ts;
trace root cause, add failing regression test,
implement minimal fix, run full relevant suite,
and report exact pass/fail commands.
```

Controlled refactor:

```
/plan Migrate <component> without API/schema changes.
Inventory callers; define invariants and rollback;
sequence small commits; propose acceptance tests.
```

Security / privacy review:

```
/security-review
Also inspect authZ, secret exposure, prompt
injection and data-exfiltration paths; classify
findings with evidence and remediation tests.
```

Codebase discovery:

```
Locate the entry point and execution path for
<business flow>. Summarize files, call graph,
state ownership and gaps. Do not edit code.
```

Operational readiness:

```
Define testable readiness: error budgets, traces,
rollback and migration checks. Implement only
instrumentation in this slice. Show measurements.
```

Doc-to-code:

```
Read @SPEC.md; extract numbered requirements.
Map each to code and a test. Report missing or
contradictory requirements before implementation.
```

Practice: For high-impact architecture or production migrations, separate discovery/specification from the implementation session (`/clear` after preserving the approved spec).

14 Testing, CI and evaluation

Quality gates: lint + typecheck + unit + contract/integration + security tests + build + explicit diff review. For UI, run and inspect rendered behavior. For agents, add adversarial tests for tool selection, leaked secrets, destructive side effects, race conditions and cost overruns.[1,6] **Constrained CI pattern:**

```
claude -p \
 "Analyze changed files for likely bugs;
  return JSON; make no edits" \
 --tools "Read,Bash" \
 --permission-mode dontAsk \
 --max-turns 6 \
 --output-format json > review.json
```

Useful metrics: pass@1, pass@k, regression rate, changed-file blast radius, test coverage delta, invalid tool calls, mean tokens/task, latency, human intervention, security violations and rollback rate. Track *observed* rather than self-reported success.[17]

Guardrail: CI must use isolated, least-privilege runners. Tool access, remote endpoints and credentials require independent configuration; `--output-format json` does not itself guarantee your desired result schema (use `--json-schema`).

15 Cost and failure diagnostics

Poor recall of constraints — Shorten CLAUDE.md; add scoped rules; check `/context`

Repeated wrong edits — Stop; reproduce; provide authoritative examples and test oracle

Large context / slow loops — Delegate search; avoid raw dumps; `/compact`; start clean task

Missing skill trigger — Improve description; verify `SKILL.md` path and frontmatter

Subagent conflicts — Assign exclusive files; use `isolation: worktree`

Permission surprise — Inspect `/permissions`, settings precedence, hook logs

MCP fails — `claude mcp list/get`; check auth and environment

Environment unstable — `/doctor`, `/status`, pinned toolchain

Practice: Optimizations: smaller tasks; smaller search outputs; shorter memory; specialized subagents; explicit completion tests; use `/usage` and `/context` before adjusting model/effort. [1,17]

16 Corrections to supplied reference

The attached reference is valuable but mixes stable practices with product-specific, rapidly changing behavior. This edition makes the following clarifications:

- `--allowedTools` is *pre-approval*; `--tools` is the built-in tool-surface restriction.[4]
- Default interactive sessions may start in `auto`; `default` is *Manual* mode, not necessarily the startup default.[1,7]
- `--bare` bypasses automatic discovery; it is not universally equivalent to a hermetic, reproducible execution environment.[4]
- `CLAUDE.md`, skills and prompts steer model behavior; hooks/permissions are the operational controls.[1,11,13]
- `npm` installation now distributes the native binary; it is not fundamentally a separate non-native runtime.[3]
- Number of hooks, exact shortcuts, agent features and command availability depend on CLI version and platform; check `/help`, `/doctor` and official reference rather than pinning unverified totals.[5,8,13]
- `/goal`, `/batch`, cloud review and background features are documented but are *not replacements* for independent test and approval gates.[5,6]

17 Source links and further reading

1. Best practices for Claude Code.
2. Claude Code overview.
3. Advanced setup / installation.
4. CLI reference.
5. Slash command reference.
6. Skills and bundled skills.
7. Permissions and modes.
8. Interactive mode and keys.
9. Prompt library.
10. Memory, CLAUDE.md and rules.
11. Skill frontmatter, arguments and eval.
12. Custom subagents.
13. Hook lifecycle reference and hook guide.
14. Sandboxed Bash and boundaries.
15. MCP integrations.
16. Plugins overview.
17. Manage costs and GitHub Actions.

Reference note: The user-supplied document "Claude Code Ultimate Reference and Engineering Cheatsheet" was analyzed and cross-checked against these official sources; disputed claims are explicitly corrected rather than replicated.